BÀI TẬP TÌM KIẾM TRONG AI

Họ và tên	Trần Đình Thi
MSSV / Lớp	3123410348 Lớp DNT1241

Mục tiêu học tập

• Mô phỏng các thuật toán BFS, DFS và Iterative Deepening.

• Biểu diễn bài toán Tháp Hà Nội dưới dạng bài toán tìm kiếm.

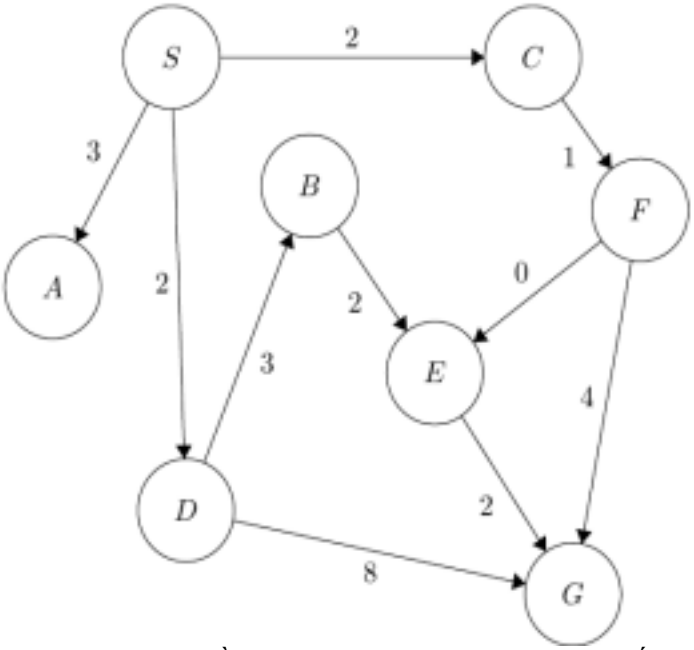
• Thiết kế và phân tích heuristic cho A*.

• So sánh DFS, BFS, UCS, Greedy Search và A* trên cùng một bài toán.

• Phân tích các phát biểu lý thuyết về heuristic, A*, Minimax và Expectimax.

Bài 1. Các thuật toán tìm kiếm

Cho đồ thị có trạng thái bắt đầu S và trạng thái đích G như hình dưới đây.



Hình 1. Đồ thị dùng cho các thuật toán tìm kiếm

Yêu cầu: Với mỗi thuật toán, hãy ghi thứ tự các nút được đưa vào explored set, đường đi được trả về và tổng chi phí của đường đi. Nếu có hòa, phá hòa theo thứ tự alphabet của trạng thái cuối cùng trong đường đi.

1.1. Breadth-First Search (BFS)

Nội dung	Kết quả

Explored Set	S, A, C, D, F, B, G
Path	S -> D -> G
Total Cost	10(2+8=10)

Giải quyết vấn đề bằng tìm kiếm **2 / 8**

1.2. Depth-First Search (DFS)

Nội dung	Kết quả
Explored Set	S, A, C, F, E, G
Path	S -> C -> F -> E -> G
Total Cost	5(2+1+0+2=5)

1.3. Iterative Deepening Search (IDS)

Giới hạn độ sâu	Các nút được duyệt
0	S
1	S,A,C,D
2	**S, A, C, F, D, B, G**
...	Thuật toán dừng tại độ sâu 2 không cần tìm độ sâu lớn hơn

Path cuối cùng: S->D->G

Total Cost: 10(2+8=10)

Câu hỏi thảo luận: BFS và DFS khác nhau ở cấu trúc frontier và tính tối ưu như thế nào?

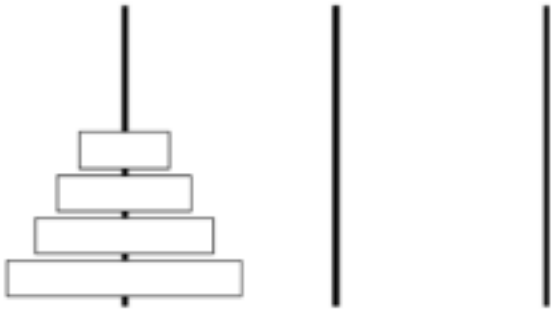
BFS và DFS khác nhau ở cấu trúc frontier và tính tối ưu như thế nào?

- Về cấu trúc frontier: BFS sử dụng hàng đợi FIFO (First-In, First-Out), duyệt theo từng mức độ sâu; DFS sử dụng ngăn xếp LIFO (Last-In, First-Out),.luôn đi sâu vào một nhánh đến khi gặp ngõ cụt mới quay lui

- Về tính tối ưu: BFS luôn tối ưu về số bước đi (tìm ra đường đi có độ sâu nhỏ nhất, và tối ưu về chi phí khi mọi chi phí bước đi bằng nhau). DFS không đảm bảo tính tối ưu, có thể tìm ra đường đi rất dài hoặc chi phí rất cao trước.

Bài 2. Tháp Hà Nội - Tower of Hanoi

Bài toán gồm n đĩa có kích thước khác nhau và ba cọc. Ban đầu các đĩa được xếp trên một cọc theo thứ tự từ lớn đến nhỏ. Mỗi lần chỉ được di chuyển một đĩa và không được đặt đĩa lớn lên trên đĩa nhỏ. Mục tiêu là chuyển toàn bộ đĩa sang cọc thứ ba.



Hình 2. Trạng thái ban đầu của bài toán Tháp Hà Nội

2.1. Biểu diễn trạng thái

Hãy đề xuất cách biểu diễn một trạng thái của bài toán. Biểu diễn phải cho biết mỗi đĩa đang ở cọc nào và đủ để xác định các hành động hợp lệ.

Biểu diễn trạng thái bằng một tuple/vector n phần tử: s = (p_1, p_2, ..., p_n) với p_i ∈ {1, 2, 3} biểu thị cọc mà đĩa i đang nằm trên đó (các đĩa đánh số 1..n từ nhỏ nhất đến lớn nhất). Do quy tắc không được đặt đĩa lớn lên trên đĩa nhỏ, trên mỗi cọc thứ tự các đĩa là duy nhất (từ nhỏ đến lớn từ trên xuống dưới), do đó vị trí p_i của mỗi đĩa là đủ để xác định hoàn toàn cấu hình bài toán.

2.2. Kích thước không gian trạng thái

Với n đĩa, hãy xác định số trạng thái có thể có theo cách biểu diễn của bạn và giải thích cách suy luận.

Với n đĩa, hãy xác định số trạng thái có thể có theo cách biểu diễn của bạn và giải thích cách suy luận:

Số trạng thái: 3^n.

Giải thích: Mỗi đĩa i ∈ {1, ..., n} có thể nằm ở 1 trong 3 cọc một cách độc lập. Do tính chất đĩa lớn ở dưới đĩa nhỏ, bất kỳ cách phân chia n đĩa vào 3 cọc đều tạo thành đúng một cách xếp hợp lệ duy nhất. Theo quy tắc nhân, ta có 3 × 3 × ... × 3 = 3^n trạng thái.

2.3. Trạng thái bắt đầu

Trạng thái bắt đầu:

S_0 = (1, 1, ..., 1) (gồm n phần tử 1, biểu thị tất cả n đĩa ban đầu đều nằm trên cọc 1).

(Hoặc biểu diễn theo danh sách cọc: S_0 = ([1, 2, ..., n], [], []))

2.4. Các hành động hợp lệ

Từ trạng thái s, Actions(s) gồm các hành động di chuyển đĩa trên cùng từ cọc nguồn i sang cọc đích j (Move(i, j) với i, j ∈ {1, 2, 3}, i ≠ j).

Điều kiện hợp lệ:

1. Cọc nguồn i phải có ít nhất một đĩa (cọc i không rỗng).
2. Chỉ được di chuyển đĩa trên cùng của cọc i (đĩa có chỉ số nhỏ nhất trong các đĩa ở cọc i).
3. Cọc đích j đang rỗng HOẶC đĩa được chuyển phải nhỏ hơn đĩa trên cùng hiện tại của cọc j (không đặt đĩa lớn lên đĩa nhỏ).

2.5. Goal Test

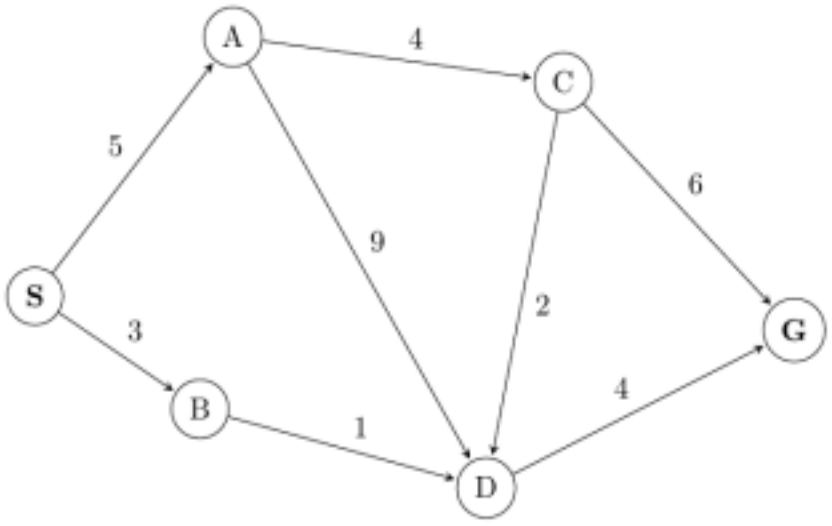
Mô tả hàm goalTest(state): Hàm trả về True khi toàn bộ n đĩa đã được chuyển sang cọc thứ 3.

Biểu diễn: s = (3, 3, ..., 3) (tất cả p_i = 3).

(Hoặc peg_3 = [1, 2, ..., n] và peg_1 = peg_2 = [])

Bài 3. Thiết kế và phân tích Heuristic

Xét đồ thị dưới đây. Hãy thiết kế heuristic và phân tích ảnh hưởng của heuristic đến thuật toán A*.



Hình 3. Đồ thị dùng để thiết kế heuristic

Công thức A*: f(n) = g(n) + h(n), trong đó g(n) là chi phí thực tế từ S đến n và h(n) là chi phí ước lượng từ n đến

đích.

3.1. Thiết kế heuristic admissible và consistent

Điền giá trị h(n) cho từng nút sao cho heuristic phù hợp với yêu cầu của nhóm.

Giải quyết vấn đề bằng tìm kiếm **4** / **8**

Node	h(n)
S	8
A	10
B	5
C	6
D	4
G	0

3.2. Chạy A* với heuristic đã thiết kế

Explored Set: S, B, D, G

- Path: S -> B -> D -> G
- Total Cost: 8 (chi phí: 3 + 1 + 4 = 8)

(Chi tiết từng bước: f(S)=8 -> sinh B(f=8), A(f=15); pop B -> sinh D(f=8); pop D -> sinh G(f=8); pop G -> đạt đích).

3.3. Heuristic admissible nhưng không consistent

Đề xuất bộ giá trị: h(S)=8, h(A)=10, h(B)=1, h(C)=6, h(D)=4, h(G)=0.
- Tính Admissible: Do h*(S)=8, h*(A)=10, h*(B)=5, h*(C)=6, h*(D)=4, h*(G)=0 nên mọi h(n) ≤ h*(n) (đặc biệt h(B)=1 ≤ 5) -> Thoả mãn admissible.
- Vi phạm Consistency: Xét cạnh S -> B có chi phí c(S, B) = 3:
  h(S) = 8 > c(S, B) + h(B) = 3 + 1 = 4. Bất đẳng thức tam giác bị vi phạm tại cạnh S -> B!

3.4. Câu hỏi mở rộng

Theo định nghĩa consistency: với mọi cạnh (u, v) có chi phí c(u, v), ta có h(u) ≤ c(u, v) + h(v).

- Giả sử đường đi tối ưu từ một nút bất kỳ n đến đích G gồm dãy các đỉnh n = n_0 -> n_1 -> n_2 -> ... -> n_k = G với tổng chi phí tối ưu là h*(n) = ∑ c(n_i, n_{i+1}).
- Áp dụng liên tiếp tính chất consistency dọc theo đường đi này:

  h(n_0) ≤ c(n_0, n_1) + h(n_1)

  h(n_1) ≤ c(n_1, n_2) + h(n_2)

  …

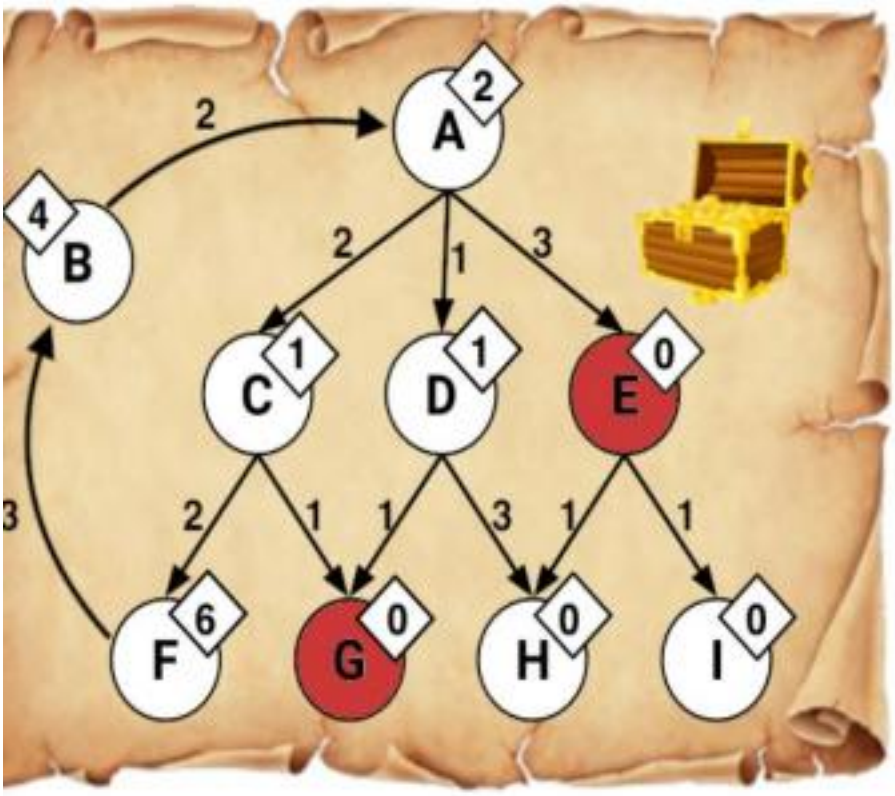
h(n_{k-1}) ≤ c(n_{k-1}, G) + h(G)

- Cộng vế theo vế (hoặc thế liên tiếp), ta được: h(n) = h(n_0) ≤ ∑ c(n_i, n_{i+1}) + h(G) = h*(n) + 0 = h*(n).
- Do đó h(n) ≤ h*(n) với mọi n, chứng minh h là admissible (đpcm).

Bài 4. Treasure Hunting (Tìm kho báu)

A là đảo xuất phát. Các nút màu đỏ là các đảo có kho báu; chỉ cần tới một trong các đảo đó. Mũi tên biểu diễn hành động có hướng; số trên cạnh là action cost. Giá trị trong hình thoi là heuristic ước lượng chi phí tối thiểu từ nút hiện tại tới một kho báu.

Giải quyết vấn đề bằng tìm kiếm **5** / **8**



Hình 4. Bản đồ tìm kho báu (giữ nguyên hình gốc)

Quy tắc phá hòa: khi lấy phần tử khỏi frontier, nếu hòa thì chọn theo thứ tự alphabet

. **4.1. Depth-First Search**

- Explored Set: A, C, F, B, G
- Path returned: A -> C -> G
- Cost: 3 (chi phí: 2 + 1 = 3)

4.2. Breadth-First Search

Explored Set:
Explored Set: A, C, D, E (nếu Goal Test khi expand khỏi frontier) hoặc {A} (nếu Early Goal Test khi vừa sinh nút)

- Path returned: A -> E
- Cost: 3

4.3. Uniform-Cost Search

- Explored Set: A, D, C, G
- Path returned: A -> D -> G
- Cost: 2 (chi phí: 1 + 1 = 2)

(Chi tiết: A sinh D(1), C(2), E(3); pop D sinh G(2), H(4); frontier có C(2) và G(2) hòa, phá hòa theo alphabet chọn C; pop C sinh F(4); pop G đạt đích).

4.4. Greedy Search

- Explored Set: A, E
- Path returned: A -> E
- Cost: 3 (chi phí A -> E là 3; E có h=0 nên được pop ngay sau A).

4.5. A* Search

Explored Set: A, D, G

- Path returned: A -> D -> G
- Cost: 2 (chi phí: 1 + 1 = 2)

(Chi tiết: A(f=2) sinh D(f=1+1=2), C(f=2+1=3), E(f=3+0=3); pop D sinh G(f=2+0=2), H(f=4); pop G đạt đích).

Bảng tổng hợp so sánh

Thuật toán	Tiêu chí chọn node	Path	Cost
DFS	Độ sâu	A->C->G	3

BFS	Độ sâu nhỏ nhất	A -> E	2
UCS	g(n)	A ->D->G	3

Greedy	h(n)	**A -> E**	2
A*	g(n)+h(n)	**A -> D -> G**	3

Bài 5. Câu hỏi Đúng/Sai

Với mỗi phát biểu, hãy ghi Đúng hoặc Sai và giải thích ngắn gọn. Nếu sai, nên đưa ra phản ví dụ.

Câu 1. Depth-first search luôn mở rộng ít nhất nhiều node bằng A* sử dụng một admissible heuristic.

Đúng / Sai: Sai

Giải thích: DFS có thể may mắn đi thẳng theo nhánh chứa đích và chỉ cần mở rộng đúng d nút (d là độ sâu của đích). Trong khi đó A* với admissible heuristic vẫn có thể phải mở rộng nhiều nhánh khác có f(n) < C*, dẫn đến mở rộng nhiều nút hơn DFS.

Câu 2. Giả sử một quân xe trong cờ vua có thể đi bất kỳ số ô theo hàng hoặc cột trong một nước đi và không thể nhảy qua quân khác. Manhattan distance là một admissible heuristic cho số nước đi nhỏ nhất từ ô A đến ô B.

Đúng / Sai: Sai

Giải thích: Quân xe có thể di chuyển nhiều ô trên cùng một hàng hoặc cột chỉ trong 1 nước đi. Ví dụ: Từ (0, 0) đến (0, 7), quân xe chỉ cần đúng 1 nước đi (h* = 1), nhưng khoảng cách Manhattan là |0-0| + |7-0| = 7. Vì h = 7 > h* = 1 nên Manhattan distance đánh giá quá cao chi phí thực tế, không admissible.

Câu 3. Euclidean distance là một admissible heuristic cho bài toán tìm đường của Pacman.

Đúng / Sai: Đúng

Giải thích: Khoảng cách Euclid là đường thẳng nối 2 điểm trên mặt phẳng, biểu diễn khoảng cách ngắn nhất giữa hai điểm trong không gian thực. Do Pacman chỉ có thể đi dọc theo các ô lưới (4 hướng) và có thể gặp tường cản, chi phí thực tế h*(n) luôn thỏa mãn h*(n) ≥ d_Manhattan ≥ d_Euclid. Do đó Euclidean distance không bao giờ đánh giá vượt quá chi phí thực tế, là một admissible heuristic.

Câu 4. Tổng của nhiều admissible heuristics vẫn luôn là một admissible heuristic.

Đúng/Sai: Sai

Giải thích: Tổng của hai heuristic admissible có thể vượt quá chi phí thực tế h*(n). Phản ví dụ: Giả sử h*(n) = 4, chọn h_1(n) = 3 và h_2(n) = 3 (cả hai đều admissible vì ≤ 4). Tổng h(n) = h_1(n) + h_2(n) = 3 + 3 = 6 > h*(n) = 4, vi phạm tính admissible. (Chỉ có max(h_1, h_2) mới luôn admissible).

Câu 5. Admissibility của một heuristic cho A* kéo theo consistency.

Đúng / Sai: Sai
Giải thích: Consistency là điều kiện ngặt hơn admissibility (Consistency ⇒ Admissibility, nhưng ngược lại thì không). Phản ví dụ: Xét đồ thị S -> A -> G với c(S, A) = 1, c(A, G) = 3 ⇒ h*(S) = 4, h*(A) = 3. Chọn h(S) = 4, h(A) = 1, h(G) = 0. Cả hai đều ≤ h* nên admissible. Nhưng tại cạnh S -> A: h(S) = 4 > c(S, A) + h(A) = 1 + 1 = 2, vi phạm điều kiện consistency.

Câu 6. A* với graph search luôn tối ưu nếu heuristic admissible.

Đúng / Sai: Sai
Giải thích: A* với Tree Search chỉ cần heuristic admissible là đảm bảo tối ưu. Tuy nhiên, A* với Graph Search (sử dụng tập explored và không duyệt lại các nút đã đóng) yêu cầu heuristic phải consistent. Nếu heuristic chỉ admissible mà không consistent, một nút có thể được đưa vào explored set qua một đường đi chưa tối ưu và sau đó bị bỏ qua khi tìm thấy đường đi tốt hơn.

Tổng kết nhanh

Nhóm	Thuật toán	Hàm/tiêu chí chính
Uninformed Search	BFS / DFS / IDS	Không sử dụng heuristic
Cost-based Search	UCS	g(n)
Informed Search	Greedy	h(n)

Giải quyết vấn đề bằng tìm kiếm **8** / **8**

Informed Search	A*	g(n)+h(n)

Ghi nhớ: A* dùng f(n) = g(n) + h(n). ---o0o---

(Hết)